



Facultad de Ciencias Exactas, Físicas y Naturales - Universidad
Nacional de Córdoba
Ingeniería en Computación

Trabajo Práctico N°1: ALU

Diseño, implementación y verificación de una ALU
parametrizable en FPGA

Arquitectura de Computadoras

Integrantes

Brigido, Tomás Andrés (Legajo 38481096)

Tosolini, Agustín (Legajo 42332698)

Docente

Pereyra, Martin

Repositorio

<https://github.com/Agustin-Tosolini/Computer-Architecture-FCEFN/tree/main/Laboratories/Lab1>

Índice

1. Introducción	3
2. Especificación	4
2.1. Repertorio de operaciones	4
2.2. Interfaz con la placa	4
2.3. Banderas de estado	5
3. Implementación de la ALU	6
3.1. Puertos y parámetros	6
3.2. Bloque secuencial: captura de los operandos	7
3.3. Bloque combinacional: el selector de operación	8
3.4. Aritmética: cómo se obtienen el acarreo y el desbordamiento	9
3.5. Desplazamientos: SRL frente a SRA	10
3.6. Salidas	11
3.7. Esquemático RTL generado	11
4. Banco de pruebas	12

1. Introducción

La *Unidad Aritmético-Lógica* (ALU, por *Arithmetic Logic Unit*) es el bloque de cómputo del camino de datos de un procesador: recibe dos operandos y un código de operación, y devuelve un resultado junto con algunas *banderas de estado* que describen propiedades de ese resultado. Prácticamente todo lo que hace una CPU con valores enteros —sumar dos registros, comparar para decidir un salto, calcular una dirección de memoria, aplicar una máscara de bits— termina pasando por ella.

El objetivo de este trabajo práctico es diseñar, describir en Verilog, verificar por simulación e implementar sobre FPGA una ALU parametrizable de 8 bits con las ocho operaciones que se listan en la Sección 2. La operación se selecciona mediante un código de 6 bits.

La placa ofrece un único bus de entrada de datos —los interruptores— que debe alcanzar para cargar dos operandos y un código de operación. La solución es multiplexarlo en el tiempo: el usuario coloca un valor en los interruptores y pulsa el botón correspondiente para guardarlo en un registro interno. Por eso el diseño no es puramente combinacional, sino un pequeño sistema síncronico con reloj, reset y registros de carga. El sistema se implementó sobre una placa **Digilent Basys 3** (FPGA Xilinx **Artix-7 XC7A35T**), con el reloj de 100 MHz que provee la placa, usando Xilinx Vivado para la síntesis, la implementación y la simulación.

2. Especificación

2.1. Repertorio de operaciones

La Tabla 1 reproduce el conjunto de operaciones exigido por el enunciado, junto con el código de 6 bits que selecciona cada una.

Operación	Código	Expresión	Tipo
ADD	100000	$A + B$	Aritmética
SUB	100010	$A - B$	Aritmética
AND	100100	$A \& B$	Lógica bit a bit
OR	100101	$A \mid B$	Lógica bit a bit
XOR	100110	$A \oplus B$	Lógica bit a bit
NOR	100111	$\sim (A \mid B)$	Lógica bit a bit
SRA	000011	$A \ggg B[2:0]$	Desplazamiento
SRL	000010	$A \gg B[2:0]$	Desplazamiento
cualquier otro código	0		Caso por defecto

Cuadro 1: Operaciones implementadas y su codificación.

Sobre esta tabla conviene hacer tres aclaraciones que se retoman más adelante:

- El **campo de desplazamiento** (*shift amount* o *shamt*) se toma de los bits bajos del operando B . La cantidad de bits usados es $\lceil \log_2 N \rceil$, donde N es el ancho de palabra: con $N = 8$ resultan 3 bits, es decir $B[2:0]$, que cubren exactamente los desplazamientos de 0 a 7 posiciones.
- El **caso por defecto** no es un detalle opcional. Los 6 bits de selección admiten $2^6 = 64$ combinaciones y sólo 8 están definidas; las 56 restantes deben producir un valor determinado —aquí, cero con todas las banderas bajas— para que el circuito sea completamente especificado y no queden inferidas memorias implícitas (*latches*).
- Todas las operaciones son de **ancho fijo**: el resultado tiene los mismos N bits que los operandos. La información que se pierde por ese truncamiento es justamente la que reportan las banderas.

2.2. Interfaz con la placa

La Tabla 2 resume los puertos del módulo y el recurso físico de la Basys 3 al que se asocia cada uno mediante el archivo de restricciones `synth-ALU.xdc`. En total son 24 puertos de entrada/salida: 13 entradas (8 interruptores, 3 pulsadores, reloj y reset) y 11 salidas (8 LEDs de resultado y 3 LEDs de bandera).

Puerto	Dir.	Recurso físico	Función
i_clk	in	Oscilador 100 MHz (W5)	Reloj del sistema
i_reset	in	Interruptor U17	Reset sincrónico activo en alto
i_switch[7:0]	in	Interruptores V17...W13	Bus de datos compartido
i_button_1	in	Pulsador W19	Carga el registro <i>A</i>
i_button_2	in	Pulsador T18	Carga el registro <i>B</i>
i_button_3	in	Pulsador T17	Carga el registro de operación
o_leds[7:0]	out	LEDs U16...V14	Resultado de la operación
o_zero	out	LED L1	Bandera <i>Z</i>
o_carry	out	LED N3	Bandera <i>C</i>
o_overflow	out	LED P1	Bandera <i>V</i>

Cuadro 2: Puertos del módulo y su asignación de pines en la Basys 3. Todos los bancos se declaran con estándar de tensión LVCMOS33.

2.3. Banderas de estado

Las tres banderas se definen sobre la operación en curso y son puramente combinatoriales: cambian en cuanto cambia cualquiera de los registros internos, sin esperar un flanco de reloj.

o_zero (Z)

Vale 1 cuando el resultado de N bits es idénticamente cero. Se calcula para *todas* las operaciones, no sólo las aritméticas. Nótese que es una bandera sobre el resultado *truncado*: si una suma produce $0x100$, los 8 bits visibles valen cero y $Z = 1$, aunque el valor matemático no lo sea. Esa aparente contradicción es precisamente lo que reporta *C*.

o_carry (C)

Es el acarreo de salida de la operación aritmética interpretada **sin signo**. En ADD indica que la suma excedió el rango $[0, 2^N - 1]$; en SUB actúa como *borrow*, es decir, vale 1 cuando $A < B$ en aritmética sin signo. Para las operaciones lógicas y de desplazamiento la bandera no tiene sentido y se fuerza a 0.

o_overflow (V)

Señala desbordamiento en la interpretación **con signo** (complemento a dos), es decir, que el resultado verdadero cae fuera del rango $[-2^{N-1}, 2^{N-1} - 1]$. Sólo se calcula para ADD y SUB.

La distinción entre *C* y *V* es uno de los puntos conceptuales centrales del práctico: el mismo circuito sumador produce los dos indicadores, pero cada uno responde a una *interpretación* distinta del mismo patrón de bits. El hardware no sabe si los operandos son con o sin signo; calcula ambas banderas y deja que el software decida cuál mirar.

3. Implementación de la ALU

El diseño completo reside en un único archivo, `ALU.v`, que describe el módulo ALU. Su estructura interna responde a un patrón muy habitual en diseño digital y conviene identificarlo antes de leer el código:

1. una **lista de puertos y parámetros** que define la interfaz;
2. un **bloque secuencial** (`always @(posedge i_clk)`) que captura las entradas en tres registros internos: `dato_A`, `dato_B` y `opt`;
3. un **bloque combinacional** (`always @(*)`) que calcula el resultado y las banderas a partir de esos registros;
4. un conjunto de **asignaciones continuas** (`assign`) que conectan las señales internas a los puertos de salida.

Esta separación estricta entre lógica secuencial y lógica combinacional no es una convención estilística: es lo que permite que la herramienta de síntesis infiera exactamente los biestables que el diseñador pretende y ni uno más.

3.1. Puertos y parámetros

```
1 module ALU#  
2 (  
3     parameter LENGTH_BITS = 8,  
4     parameter LENGTH_OPT = 6  
5 )  
6 (  
7     output wire [LENGTH_BITS-1 : 0] o_leds,  
8     output wire o_zero,  
9     output wire o_carry,  
10    output wire o_overflow,  
11  
12    input wire i_clk,  
13    input wire i_reset,  
14    input wire [LENGTH_BITS-1:0] i_switch,  
15    input wire i_button_1,  
16    input wire i_button_2,  
17    input wire i_button_3  
18 );
```

Código 1: Declaración del módulo: parámetros y puertos.

El módulo se declara con **parámetros**: `LENGTH_BITS` fija el ancho de palabra (8 bits) y `LENGTH_OPT` el ancho del código de operación (6 bits). Un `parameter` es una constante de *tiempo de elaboración*: la herramienta la resuelve antes de generar hardware, de modo que expresiones como `[LENGTH_BITS-1:0]` se convierten en rangos fijos durante la síntesis. No hay coste en área por parametrizar; lo que se obtiene es un módulo reutilizable con otro ancho sin tocar una línea del cuerpo.

Todos los puertos se declaran `wire`, lo cual es coherente con el hecho de que las salidas se manejan con asignaciones continuas. El estado interno se declara aparte:

```

1  reg [LENGTH_BITS-1 : 0] result;
2  reg carry;
3  reg overflow;
4
5  reg [LENGTH_BITS-1 : 0] dato_A = {LENGTH_BITS{1'b0}};
6  reg [LENGTH_BITS-1 : 0] dato_B = {LENGTH_BITS{1'b0}};
7  reg [LENGTH_OPT-1 : 0] opt = {LENGTH_OPT{1'b0}};
8
9  localparam [5:0] ADD = 6'b100000,
10                  SUB = 6'b100010,
11                  AND_ = 6'b100100,
12                  OR_ = 6'b100101,
13                  XOR_ = 6'b100110,
14                  NOR_ = 6'b100111,
15                  SRA = 6'b000011,
16                  SRL = 6'b000010;
17
18  localparam SHAMT_BITS = $clog2(LENGTH_BITS);

```

Código 2: Señales internas y codificación de las operaciones.

3.2. Bloque secuencial: captura de los operandos

```

1  always @(posedge i_clk) begin
2      if (i_reset) begin
3          dato_A <= {LENGTH_BITS{1'b0}};
4          dato_B <= {LENGTH_BITS{1'b0}};
5          opt <= {LENGTH_OPT{1'b0}};
6      end else if (i_button_1) dato_A <= i_switch;
7      else if (i_button_2) dato_B <= i_switch;
8      else if (i_button_3) opt <= i_switch[LENGTH_OPT-1:0];
9  end

```

Código 3: Registros de entrada con reset sincrónico y carga por prioridad.

Este bloque es el corazón secuencial del diseño y concentra cuatro decisiones importantes.

Sensibilidad sólo al flanco de reloj. La lista de sensibilidad es `@(posedge i_clk)` y nada más. Como `i_reset` no aparece en ella, el reset es **sincrónico**: su efecto se materializa recién en el siguiente flanco ascendente. En las FPGA de la serie 7 esta es la opción preferida, porque el biestable primitivo `FDRE` posee una entrada de reset sincrónico dedicada; un reset asíncrono obligaría a usar la primitiva `FDCE` y, además, introduce el riesgo de violar el tiempo de recuperación (*recovery time*) al liberarse en un instante arbitrario.

Asignaciones no bloqueantes. Dentro de un bloque sincrónico se usa `<=`. La semántica es que *todas* las lecturas del lado derecho se evalúan con los valores previos al flanco y las escrituras se aplican al final del paso de simulación. Esto modela fielmente lo que hace un conjunto de biestables que conmutan simultáneamente y evita las condiciones de carrera que aparecerían con `=` si un registro alimentara a otro.

Cadena if / else if: prioridad, no paralelismo. La estructura condicional encadenada establece un orden estricto de prioridad:

```
i_reset > i_button_1 > i_button_2 > i_button_3
```

Si se presionan dos botones a la vez, sólo se atiende el de mayor prioridad. Esta descripción no genera un multiplexor gigante: como cada rama escribe un registro distinto y las ramas no cubren todos los casos, la herramienta infiere una **habilitación de reloj** (*clock enable*) por registro. Concretamente, dato_B se carga si y sólo si $\sim i_reset \& \sim i_button_1 \& i_button_2$. El coste es prácticamente nulo, porque cada biestable de un *slice* Artix-7 ya trae un pin CE disponible. En el esquemático esto se ve como los bloques RTL_REG_SYNC con pines CE y RST, alimentados por unos pocos RTL_MUX de dos entradas.

No hay else final. Cuando ninguna condición se cumple, ningún registro cambia: retienen su valor. En un bloque sincrónico eso es exactamente lo deseado (un biestable que no recibe habilitación conserva su estado) y no genera hardware indeseado. Es importante notar que esta regla *no* se aplica a los bloques combinacionales, donde la ausencia de una rama sí produce un *latch*.

Truncamiento del código de operación. La expresión `i_switch[LENGTH_OPT-1:0]` toma sólo los 6 bits bajos de los 8 interruptores. Los 2 bits superiores se descartan de forma deliberada, lo que implica que los patrones 0x22 y 0xE2, por ejemplo, seleccionan la misma operación (SUB). El banco de pruebas verifica explícitamente este comportamiento.

3.3. Bloque combinacional: el selector de operación

```
1  always @(*) begin
2      result = {LENGTH_BITS{1'b0}};
3      carry = 1'b0;
4      overflow = 1'b0;
5
6      case (opt)
7          ADD: begin
8              {carry, result} = {1'b0, dato_A} + {1'b0, dato_B};
9              overflow = (dato_A[LENGTH_BITS-1] == dato_B[LENGTH_BITS-1]) &&
10                 (result[LENGTH_BITS-1] != dato_A[LENGTH_BITS-1]);
11          end
12          SUB: begin
13              {carry, result} = {1'b0, dato_A} - {1'b0, dato_B};
14              overflow = (dato_A[LENGTH_BITS-1] != dato_B[LENGTH_BITS-1]) &&
15                 (result[LENGTH_BITS-1] != dato_A[LENGTH_BITS-1]);
16          end
17          AND_: result = dato_A & dato_B;
18          OR_: result = dato_A | dato_B;
19          XOR_: result = dato_A ^ dato_B;
20          NOR_: result = ~(dato_A | dato_B);
21          SRA: result = $signed(dato_A) >>> dato_B[SHAMT_BITS-1:0];
22          SRL: result = dato_A >> dato_B[SHAMT_BITS-1:0];
23          default: result = {LENGTH_BITS{1'b0}};
24      endcase
25  end
```

Código 4: Núcleo combinacional de la ALU.

La lista de sensibilidad implícita. `always @(*)` le indica a la herramienta que construya automáticamente la lista con todas las señales leídas dentro del bloque (`opt`, `dato_A`, `dato_B`). Escribirla a mano es una fuente clásica de errores: si se omite una señal, el simulador no reevalúa el bloque cuando esa señal cambia y aparece una discrepancia entre simulación y hardware sintetizado.

Asignaciones por defecto: la defensa contra los *latches*. Las tres primeras líneas asignan un valor a `result`, `carry` y `overflow` *antes* de entrar al `case`. Esto garantiza que, sea cual sea el camino recorrido, las tres señales reciben un valor en toda ejecución del bloque. Si no fuera así —por ejemplo, si `carry` sólo se asignara en las ramas `ADD` y `SUB`— Verilog exigiría que la señal *recuerde* su valor anterior en las demás ramas, y la síntesis inferiría un *latch* transparente. Un *latch* en una FPGA es casi siempre un error: introduce caminos sensibles al nivel, complica el análisis de temporización y puede generar realimentaciones combinacionales. La regla práctica es: **en todo bloque combinacional, asignar valores por defecto al inicio.**

Asignaciones bloqueantes. Aquí se usa `=`, que ejecuta de forma inmediata y secuencial, como en un lenguaje de programación. Esto no es opcional: la línea que calcula `overflow` *lee* `result`, y necesita ver el valor recién calculado por la suma. Con asignaciones no bloqueantes, `result` todavía tendría el valor por defecto (cero) y la bandera saldría mal. La regla general —`<=` en bloques sincrónicos, `=` en bloques combinacionales— se respeta estrictamente en este diseño.

El `case` es un multiplexor, no un salto. Este es probablemente el concepto más importante del práctico. Al leer el código uno tiende a imaginar que sólo se «ejecuta» la rama seleccionada, pero el hardware no funciona así: el sintetizador instancia **todos** los operadores simultáneamente y coloca un multiplexor a la salida, gobernado por `opt`. En todo momento el sumador está sumando, el restador está restando y los desplazadores están desplazando; lo único que decide `opt` es cuál de esos resultados llega a `result`. De ahí que el área del circuito sea la *suma* de todas las operaciones y que el retardo crítico sea el de la rama más lenta más el del multiplexor. Esto se aprecia con toda claridad en el esquemático de la Figura 1.

El caso por defecto. La rama `default` cubre las 56 combinaciones no asignadas y devuelve cero con las banderas bajas. Además de dar un comportamiento definido, su presencia le confirma a la herramienta que el `case` es completo, lo que evita recurrir a directivas como `full_case`.

3.4. Aritmética: cómo se obtienen el acarreo y el desbordamiento

Extracción del acarreo. La construcción

```
{carry, result} = {1'b0, dato_A} + {1'b0, dato_B};
```

es un modismo estándar. Ambos operandos se extienden con un cero a la izquierda hasta $N + 1 = 9$ bits, la suma se realiza con ese ancho y el resultado se reparte por concatenación: los 8 bits bajos van a `result` y el noveno, que es precisamente el acarreo de salida, va a `carry`. La extensión con cero (y no con el bit de signo) indica que la operación se interpreta **sin signo**, que es lo que corresponde a la bandera C.

Vale aclarar que en Verilog-2001 el ancho de la operación del lado derecho también queda determinado por el contexto de la asignación, de modo que escribir `{carry, result} = dato_A + dato_B` daría el mismo resultado. Se prefiere la forma explícita porque documenta la intención y no depende de las reglas —bastante sutiles— de propagación de anchos del lenguaje.

En SUB, el mismo mecanismo aplicado a una resta de 9 bits hace que `carry` funcione como **indicador de préstamo** (*borrow*): si $A \geq B$ el resultado de 9 bits es positivo y el bit 8 vale 0; si $A < B$ el resultado es negativo en complemento a dos y el bit 8 vale 1.

Detección del desbordamiento con signo. El desbordamiento no puede leerse de un solo bit: se deduce comparando los signos de los operandos con el signo del resultado.

Para la **suma**, dos números de signos opuestos nunca pueden desbordar (el resultado queda acotado entre ambos). El desbordamiento sólo es posible cuando ambos tienen el mismo signo, y se confirma si el resultado cambia de signo:

$$V_{\text{ADD}} = (A_{N-1} = B_{N-1}) \wedge (R_{N-1} \neq A_{N-1})$$

Por ejemplo, con 8 bits: $0x7F + 0x01 = 0x80$, es decir $127 + 1 = -128$. Ambos operandos son positivos y el resultado es negativo: $V = 1$. Nótese que en ese caso $C = 0$, lo que ilustra que las dos banderas son independientes.

Para la **resta**, el razonamiento se invierte: $A - B$ sólo puede desbordar si A y B tienen signos distintos, y se confirma si el resultado no conserva el signo de A :

$$V_{\text{SUB}} = (A_{N-1} \neq B_{N-1}) \wedge (R_{N-1} \neq A_{N-1})$$

Ambas expresiones son paramétricas: al usar el índice `LENGTH_BITS-1` para referirse al bit de signo, siguen siendo válidas si se cambia el ancho de palabra.

3.5. Desplazamientos: SRL frente a SRA

```
SRA: result = $signed(dato_A) >> dato_B[SHAMT_BITS-1:0];
SRL: result = dato_A >> dato_B[SHAMT_BITS-1:0];
```

La diferencia entre ambos es el relleno que entra por la izquierda:

- $\gg$ es el **desplazamiento lógico**: introduce ceros. Corresponde a una división entera sin signo por 2^k .
- $\gg$ es el **desplazamiento aritmético**: replica el bit de signo, preservando el valor con signo. Corresponde a una división con signo por 2^k (con redondeo hacia $-\infty$).

3.6. Salidas

```
1 assign o_leds = result;
2 assign o_zero = (result == {LENGTH_BITS{1'b0}});
3 assign o_carry = carry;
4 assign o_overflow = overflow;
```

Código 5: Asignaciones continuas de salida.

Las cuatro salidas se conectan con asignaciones continuas, que describen conexiones permanentes: el valor del lado izquierdo sigue al del derecho en todo momento.

3.7. Esquemático RTL generado

La Figura 1 muestra el esquemático RTL que Vivado elabora a partir de ALU.v. Se trata de una vista previa al mapeo tecnológico.

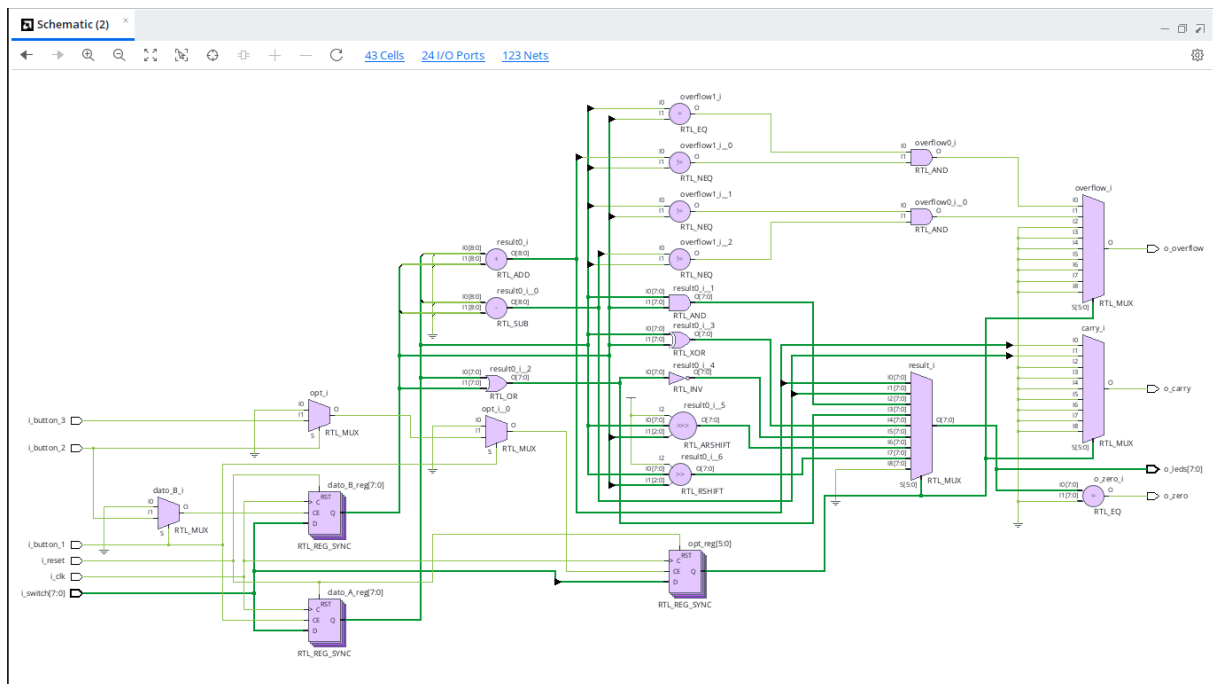


Figura 1: Esquemático RTL generado por Vivado a partir de ALU.v: 43 celdas, 24 puertos de entrada/salida y 123 conexiones.

4. Banco de pruebas

La verificación del diseño se realizó con un banco de pruebas **autoverificable** escrito en Verilog (ALU_tb.v, dentro del conjunto de fuentes de simulación del proyecto). «Autoverificable» significa que el propio banco contiene un *modelo de referencia* que calcula, a partir de la especificación, el resultado y las tres banderas que deberían observarse, y los compara automáticamente contra las salidas del módulo bajo prueba. El operador no tiene que inspeccionar formas de onda para decidir si el diseño funciona: la simulación termina informando la cantidad de verificaciones realizadas y de errores detectados.

El banco instancia la ALU con los mismos parámetros del diseño (8 bits de datos, 6 de operación), genera un reloj de 100 MHz idéntico al de la placa y aplica los estímulos a través de tareas que respetan el protocolo real de carga (levantar el botón, esperar al menos un flanco, bajarlo). Cubre, entre otros aspectos, el reset sincrónico, la carga individual de cada registro, la prioridad entre pulsadores, el truncamiento del código de operación, un barrido exhaustivo de las 64 combinaciones posibles de opt, casos borde por operación (acarreo, desbordamiento, desplazamientos de números negativos) y un bloque final de vectores aleatorios con reporte de cobertura por operación. En esta sección no se detalla su implementación; se presentan únicamente las capturas que documentan su ejecución.

Las Figuras 2 y 3 muestran las formas de onda de la simulación conductual en Vivado. Junto a las señales de la interfaz aparecen las señales internas del banco: m_A, m_B y m_opt son el modelo espejo de los registros del diseño, y checks y errors son los contadores de verificaciones y de fallos.

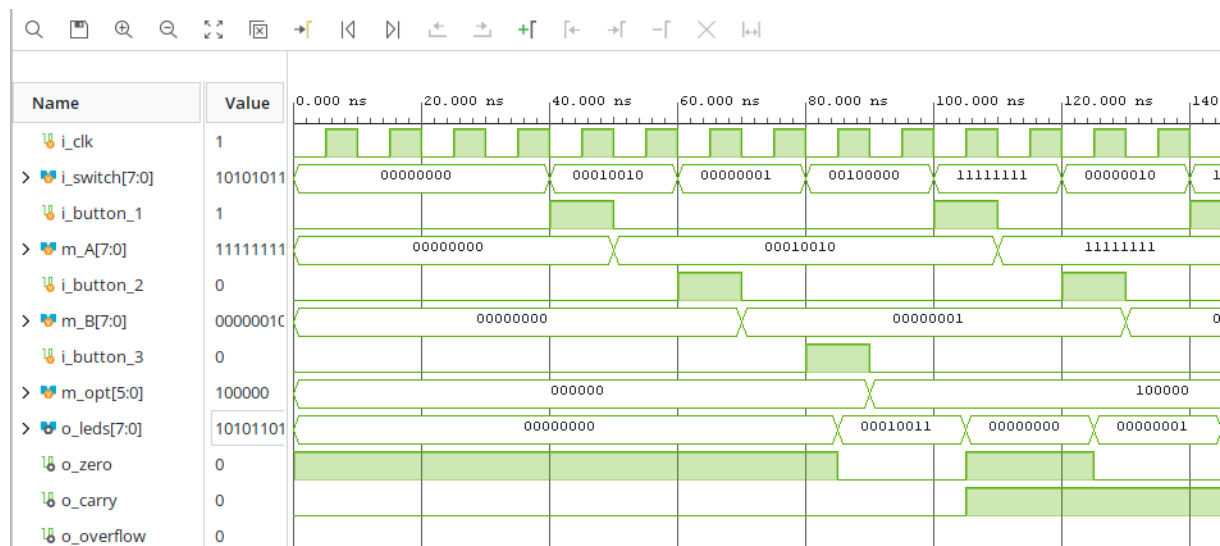


Figura 2: Simulación: reset inicial y carga sucesiva de los registros.

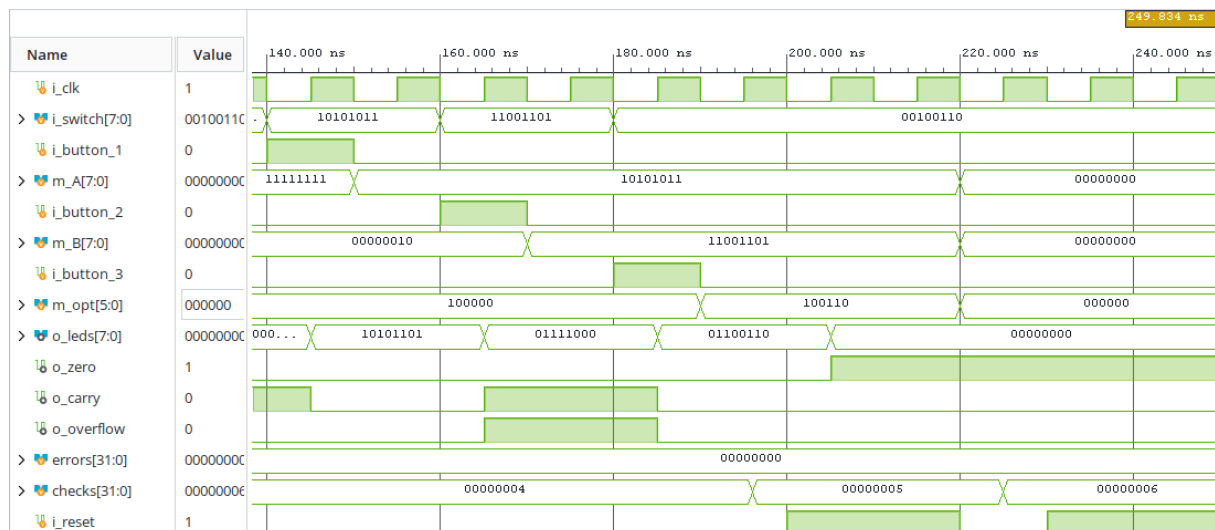


Figura 3: Simulación: Carga de valores en A, B y opt para posterior reset.

Finalmente, la Figura 4 reproduce el informe que el banco imprime por consola al terminar: el recuento de veces que se ejercitó cada operación válida durante la etapa aleatoria —útil para confirmar que ninguna quedó sin probar— y el resumen final, que incluye la semilla utilizada (necesaria para reproducir la corrida), el total de verificaciones y el número de errores.

```

--- COBERTURA de opcodes validos ---
  ADD (100000) : 61 veces
  SUB (100010) : 62 veces
  AND (100100) : 57 veces
  OR  (100101) : 61 veces
  XOR (100110) : 61 veces
  NOR (100111) : 61 veces
  SRA (000011) : 59 veces
  SRL (000010) : 67 veces

=====
RESUMEN
Semilla      : 20260830
Verificaciones : 620
Errores      : 0
RESULTADO    : *** TODOS LOS TESTS PASARON ***
=====

```

Figura 4: Reporte de cobertura y resumen final impresos por el banco de pruebas. En la corrida capturada se ejecutaron 620 verificaciones sin errores, con las ocho operaciones válidas ejercitadas.

El resultado ***** TODOS LOS TESTS PASARON ***** constituye la evidencia de que la implementación descrita en la Sección 3 satisface la especificación de la Sección 2 para todos los estímulos aplicados.